
Development of an Intelligent Assistant for Data Preparation Automation in Urban Transportation Management Systems

Thesis for the Award of the Degree of
MASTER
7M06105 — Computer Science and Engineering

By
RAMAZAN BAKYTULY

Under the Supervision of
Aivar Sakhipov Aituarovich



SCHOOL OF SOFTWARE ENGINEERING
ASTANA IT UNIVERSITY LLP
ASTANA, KAZAKHSTAN
JUNE 2026

Abstract

Urban transportation systems produce heterogeneous traffic and sensor data that must be cleaned, validated, and prepared before analysis. This thesis presents an intelligent assistant for automating that preparation and extends it into **Flowmatic**, a deployable platform for batch uploads and smart-city pipelines.

The classical study combines multi-dimensional data quality assessment, automated feature engineering, ensemble anomaly detection, and gradient-boosted event classification. On semi-synthetic Astana traffic data, the full preparation pipeline reaches 96.83% accuracy ($F1 = 0.968$) with XGBoost, clearly above raw-data and generic AutoML baselines.

Flowmatic implements the same preparation logic in a NestJS/Vue stack orchestrated with Docker Compose. Batch uploads pass through quality checks and cleaning workers backed by PostgreSQL and S3-compatible storage. A smart-city workbench connects an Astana geospatial traffic simulator, Manual/Auto model routing, and medallion lake export. Eight neural models covering forecasting, anomaly reconstruction, imputation, graph traffic, and severity classification are evaluated on held-out temporal test splits with three random seeds; the severity classifier reaches macro-F1 = 0.911 ± 0.017 . Model weights are published on Hugging Face together with reproducible training scripts.

The dissertation links classical preparation research with a working multimodal platform, providing auditable quality metrics, modality-aware inference, and open experimental artifacts.

Keywords: Data Preparation, Intelligent Transportation Systems, Smart Cities, Model Routing, Time Series, Anomaly Detection, Flowmatic

Dedication and acknowledgements

I dedicate this work to my family and to the faculty and colleagues at Astana IT University who supported this research.

I acknowledge the use of open traffic benchmarks, Hugging Face model hosting, and the Flowmatic open-source platform developed during this thesis.

Student's declaration

I, Ramazan Bakytuly certify that the work embodied in this research/project work, carried out by me under the supervision of Aivar Sakhipov Aituarovich for the period of September 2023 to June 2026 at Astana IT University. The work has not been submitted for the award of any other degree/ diploma except where due acknowledgement has been made in the text.

I further declare that the material obtained from other sources has been duly acknowledged in the thesis.

SIGNED: DATE:

Certificate by Supervisor

This is to certify that research work embodied in this thesis entitled “Development of an Intelligent Assistant for Data Preparation Automation in Urban Transportation Management Systems” submitted to Astana IT University, for the award of the degree of Master (7M06105 Computer Science and Engineering) has been carried out by Ramazan Bakytuly under my supervision at Astana IT University, Astana from September 2023 to June 2026.

To the best of my knowledge and belief, this work is original and has not been submitted so far in part or in full for the award of any degree or diploma of any University/ Institute.

SIGNED: DATE:

List of Tables

Table	Page
4.1 End-to-end batch upload experiment on <code>astana_synthetic_data.csv</code> via Flowmatic production API (May 2026).	20
4.2 Comparison of classical baselines and Flowmatic neural checkpoints on Astana traffic data.	20
4.3 Smart-city live demo pipeline (<i>Astana Live Demo</i>) with simulated sensor sources.	20
4.4 Manuscript versus implementation coverage (selected dimensions).	21
5.1 Experimental datasets used in Flowmatic Phase 2–3 evaluation (May 2026).	23
5.2 Temporal data split protocol (identical across neural runs).	23
5.3 Production portfolio: multi-seed test metrics (seeds 42, 7, 2026; mean $\pm$ 95% CI).	23
5.4 Sequence-length ablation (Astana PatchTST, ETT DLinear; test RMSE).	24
5.5 Cross-dataset transfer (train on source, evaluate on target test split; RMSE).	24

List of Figures

Figure	Page
4.1 Batch upload pipeline (NestJS + Vue).	15
4.2 Smart-city pipeline with adaptive core unit and medallion lake.	15
4.3 Docker Compose deployment topology.	16
4.4 Medallion data lake layout (bronze, silver, gold).	17
4.5 Flowmatic login interface.	18
4.6 Astana Geospatial Demo workbench with simulated traffic and weather sources.	18
4.7 Adaptive core unit configuration with Manual/Auto routing and Hugging Face model catalogue.	19
4.8 Workflow graph for the Astana geospatial simulator pipeline.	19
5.1 Forecasting and reconstruction leaderboard (test RMSE).	26
5.2 Multi-seed test RMSE dispersion for production portfolio models.	27
5.3 Sequence-length ablation ($L = 24$ vs $L = 96$).	27
5.4 Cross-dataset transfer (train source $\rightarrow$ test target).	28
5.5 Streaming inference latency vs batch size (TorchScript, GPU).	28
5.6 Streaming throughput (events/s) for production checkpoints.	29
5.7 Validation loss curves during neural model training (representative runs). . .	29
5.8 Severity classifier macro-F1 compared with classical baselines.	30

Table of Contents

Abstract	i
Dedication and acknowledgements	ii
Student’s declaration	iii
Certificate by Supervisor	iv
List of Tables	v
List of Figures	vi
Table of Contents	vii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Research Objectives	2
1.4 Research Questions	2
1.5 Contributions	3
1.6 Scope and Limitations	3
1.7 Thesis Structure	3
2 Literature Review	4
2.1 Scope of the Review	4
2.2 Data Quality in Transportation Analytics	4
2.3 Automated Preprocessing and Missing Data	5
2.4 Feature Engineering for Traffic Data	5
2.5 Anomaly Detection and Event Classification	6

2.6	Intelligent Transportation Systems and Deployment	7
2.7	Explainability and Research Gap	7
2.8	Neural Time-Series Models in Traffic Analytics	7
2.9	Scope of This Thesis	8
3	Methodology	9
3.1	Overview	9
3.2	Data Quality Assessment	9
3.3	Classical Preparation Pipeline	9
3.4	Adaptive Core Unit and Model Registry	10
3.4.1	Registry	10
3.4.2	Routing policy	10
3.5	Neural Task Formulations	11
3.5.1	Forecasting (PatchTST, iTransformer, DLinear, TimesBlock, STGCN)	11
3.5.2	Anomaly detection (TranAD-style reconstruction)	11
3.5.3	Imputation (SAITS-style masked reconstruction)	11
3.5.4	Severity classification (Transformer encoder)	12
3.6	Neural Evaluation Methodology	12
3.7	Insight Engine and Copilot	12
4	Implementation	13
4.1	System Architecture	13
4.2	Batch Preparation Module	13
4.3	Smart City Module	13
4.4	Model Inference Service	14
4.5	Research Artifact Pipeline	14
4.6	User Interface	14
4.7	Implementation Summary	19
5	Results and Discussion	22
5.1	Overview	22
5.2	Experimental Protocol	23
5.2.1	Datasets	23
5.2.2	Hold-out validation	24
5.2.3	Multi-seed robustness	24
5.3	Classical Preparation Results	24

5.4	Platform Case Studies	25
5.4.1	Batch upload experiment	25
5.4.2	Live smart-city workbench	25
5.5	Neural Production Portfolio	25
5.5.1	Model definitions	25
5.5.2	Multi-seed test performance	25
5.6	Ablations and Generalisation	30
5.6.1	Sequence length	30
5.6.2	Cross-dataset transfer	30
5.6.3	Streaming inference latency	30
5.7	Comparison with Classical Baselines	30
5.8	Discussion	31
5.9	Summary	31
6	Conclusion and Future Directions	32
6.1	Summary of Contributions	32
6.2	Key Findings	32
6.3	Limitations	33
6.4	Future Work	33
6.5	Concluding Remarks	33
A	Reproducibility and Published Artifacts	34
A.1	Repository Commands	34
A.2	Model Objectives	34
	Bibliography	35

Chapter 1

Introduction

1.1 Background and Motivation

Urban transportation management systems (UTMS) depend on timely, reliable sensor and event data. In practice, agencies receive CSV exports, API feeds, and streaming telemetry with missing values, inconsistent schemas, duplicate records, and domain-specific anomalies. Manual preparation is slow, difficult to audit, and hard to reproduce when models or analysts change.

Research on data quality [1, 2], automated preprocessing [3, 4], and intelligent transportation systems [5, 6] shows that preparation should be treated as a first-class engineering problem rather than an ad hoc notebook step. This thesis addresses that gap for urban traffic analytics by designing, implementing, and evaluating an intelligent preparation assistant that later evolved into the deployable **Flowmatic** platform.

1.2 Problem Statement

The central problem is how to automate data preparation for urban traffic data while preserving auditability and supporting both batch analytics and operational smart-city pipelines. Existing work often studies quality assessment, imputation, feature engineering, or anomaly detection in isolation. Operational teams still lack a unified workflow that records quality diagnostics, applies domain-aware transformations, and connects prepared data to downstream models in a reproducible way.

1.3 Research Objectives

The thesis pursues five objectives aligned with the completed work:

1. To design and evaluate an automated data preparation workflow for urban traffic data, including multi-dimensional quality assessment, feature generation, ensemble anomaly screening, and tabular event classification.
2. To implement that workflow as a containerised web platform with authenticated upload, asynchronous processing, durable storage, and export to common destinations.
3. To extend the platform with a smart-city pipeline that ingests simulated or external sensor streams—including an Astana geospatial traffic simulator—routes events through an adaptive core unit, and persists outputs in a medallion-style data lake.
4. To train and evaluate a portfolio of neural models for forecasting, anomaly reconstruction, imputation, and severity classification on held-out temporal test splits with multi-seed replication.
5. To validate the deployed system through reproducible API experiments and a live Astana smart-city demonstration suitable for thesis defence.

1.4 Research Questions

- RQ1:** Does domain-aware automated preparation improve tabular traffic event classification compared with raw data and generic feature automation?
- RQ2:** Can the preparation workflow be operationalised as a reproducible service with measured end-to-end latency and quality reporting?
- RQ3:** How do neural time-series models perform across forecasting, reconstruction, imputation, and classification tasks under a common temporal hold-out protocol?
- RQ4:** Does a registry-based Manual/Auto routing policy allow modality-appropriate model selection in a smart-city pipeline?

1.5 Contributions

The main contributions are:

- a six-dimensional Data Quality Index and quality-guided preparation pipeline evaluated on Astana and public traffic benchmarks;
- the Flowmatic platform implementing batch preparation and smart-city pipelines in a Docker Compose stack;
- an adaptive core unit with a documented model registry and Manual/Auto routing rules;
- an eight-model neural portfolio with multi-seed held-out test metrics and published Hugging Face checkpoints;
- reproducible platform experiments, including batch upload timing and a live Astana geospatial simulator demo.

1.6 Scope and Limitations

The classical experiments use semi-synthetic Astana data and public benchmarks (METR-LA, PEMS-BAY, PeMSD4). The platform demo uses simulated sensor feeds rather than a production agency deployment. Router oracle ablations and Astana-trained external hold-out transfer are reported as future work.

1.7 Thesis Structure

Chapter 2 reviews related work. Chapter 3 describes the preparation methodology and neural evaluation protocol. Chapter 4 presents the Flowmatic implementation. Chapter 5 reports classical, platform, and neural results. Chapter 6 summarises findings and future directions. Appendix A lists reproducibility commands and artifacts.

Chapter 2

Literature Review

2.1 Scope of the Review

This chapter reviews the literature needed for an intelligent assistant that automates data preparation in urban transportation management systems. The review is intentionally focused on four questions that shape the proposed system: how traffic data quality should be assessed, how missing and noisy traffic observations should be treated, how spatiotemporal features can be generated automatically, and how anomaly detection and explainability support operational use.

2.2 Data Quality in Transportation Analytics

General data quality research treats quality as fitness for use rather than a fixed property of a dataset. Wang and Strong [1] group quality into intrinsic, contextual, representational, and accessibility dimensions, while Pipino et al. [2] show that useful assessment combines objective metrics with user-oriented judgments. In transportation systems this perspective is especially important because the same sensor record may be acceptable for strategic planning but unusable for real-time incident response.

Transportation data adds domain-specific constraints. Sensor streams are spatially distributed, time ordered, and often produced by heterogeneous devices. Rahm and Do [7] describe common quality problems in integrated data sources, and Heinrich et al. [8] connect those problems to intelligent transportation systems, where positional accuracy, temporal consistency, and network coverage are central. Composite indices such as the

Data Quality Index [9] provide a concise way to aggregate dimensions, but the weighting must remain application dependent. For this thesis, the quality score therefore uses six dimensions: completeness, consistency, accuracy, timeliness, uniqueness, and validity.

2.3 Automated Preprocessing and Missing Data

Automated preprocessing research has moved from fixed rule chains toward pipelines that adapt to data characteristics and downstream tasks. Liu et al. [3] describe automated preprocessing as a pipeline optimization problem, while scenario-based preprocessing for traffic accident severity prediction shows the value of choosing operations from domain context rather than applying generic defaults [10]. AutoML work also shows that preprocessing and feature construction should be treated as part of the model search space rather than as isolated manual steps [4, 11].

Missing traffic data is a recurring operational problem caused by sensor failures, communication gaps, and maintenance outages. Recent reviews emphasize that effective imputation must use temporal and spatial structure instead of relying only on mean or median replacement [12, 13]. Tensor methods, graph-based models, autoencoders, and probabilistic approaches are frequently used, but they differ in data requirements and deployment cost. Boquet et al. [14] show that variational autoencoders can support imputation, dimensionality reduction, model selection, and anomaly detection within a single traffic forecasting framework. For a practical assistant, this motivates a modular design: simple methods remain available for small or sparse datasets, while spatiotemporal and ensemble methods are selected when enough historical structure exists.

2.4 Feature Engineering for Traffic Data

Feature engineering is a major source of performance improvement in traffic analytics because raw timestamps, coordinates, and sensor counts rarely expose the periodic and network effects needed by learning algorithms. Automated feature generation has been studied for urban management and traffic forecasting [15, 16]. Recent spatiotemporal forecasting literature shows that temporal cycles, local neighborhood relationships, and learned graph structure all affect prediction quality [17–19].

The literature suggests a useful division between interpretable engineered features and representation learning. Deep graph and Transformer models can learn complex

dependencies [20, 21], but operational systems still benefit from explicit features such as peak-hour indicators, rolling statistics, density proxies, speed deviations, and distance-to-center measures. Such features are easier to audit, cheaper to compute, and suitable for classical classifiers. The proposed assistant therefore generates a compact set of traffic-aware features and then selects among them using mutual information and model-based importance.

2.5 Anomaly Detection and Event Classification

Traffic anomaly detection covers both data quality errors and operational events. Classical methods such as Local Outlier Factor [22] and Isolation Forest [23] remain attractive because they are efficient and interpretable. Broader anomaly detection surveys [24, 25] distinguish point, contextual, and collective anomalies, which maps well to transportation data: a speed may be normal on one road at midnight but anomalous on another road during peak hours.

Recent traffic studies extend this baseline with deep, streaming, and federated approaches. Variational and autoencoder-based models can learn normal traffic patterns from multivariate time series [26, 27]; real-time urban traffic work emphasizes low-latency anomaly detection and load balancing [28]; and hierarchical federated learning has been proposed for trajectory anomaly detection across regions [29]. These studies support the thesis decision to use an ensemble detector: no single method is robust across sparse, noisy, and context-dependent anomalies.

For event classification, Random Forests, gradient boosting, and support vector machines remain strong baselines [30–32]. Gradient boosting is particularly suitable because it handles nonlinear feature interactions and missing values efficiently, while Random Forests provide stable feature importance. This thesis evaluates multiple classifiers but uses XGBoost as the principal downstream model when comparing preprocessing strategies.

2.6 Intelligent Transportation Systems and Deployment

Intelligent transportation systems are increasingly built around IoT sensing, event streams, and distributed analytics. Smart city architectures rely on sensing, communication, data management, and application layers [6]; big-data ITS surveys identify volume, velocity, variety, and veracity as persistent challenges [5]; and Kafka-style event streams provide a practical foundation for real-time data pipelines [33, 34]. These works indicate that data preparation should not be a one-time offline script. It must be deployable as a service that can validate, enrich, and score data continuously.

2.7 Explainability and Research Gap

Explainability matters because transportation agencies need to justify alerts, feature choices, and model outputs. SHAP values provide a consistent feature attribution method for tree and ensemble models [35, 36], while broader interpretability work warns that explanations must be tied to concrete user decisions rather than presented as decorative model outputs [37, 38].

The reviewed literature leaves a practical integration gap. Data quality assessment, imputation, feature engineering, anomaly detection, classification, streaming deployment, and explainability are often studied separately. This thesis addresses that gap by implementing a unified assistant that connects these stages into one reproducible pipeline and evaluates whether automated preparation improves downstream traffic event classification.

2.8 Neural Time-Series Models in Traffic Analytics

Recent work on traffic forecasting uses decomposition linear models [39], patch-based Transformers [40], and graph spatio-temporal networks [41]. Deployed smart-city systems typically combine stream ingestion, model registries, and latency-aware inference rather than offline notebooks alone [5, 34]. Flowmatic follows this pattern: classical preparation provides tabular baselines, while neural checkpoints handle modality-specific forecasting, reconstruction, and classification tasks selected by explicit routing rules.

2.9 Scope of This Thesis

The dissertation reports two linked parts that were actually completed:

1. a classical automated preparation study with ensemble anomaly detection and tree-based classification on Astana traffic data;
2. a production implementation (**Flowmatic**) with neural extensions, adaptive routing, and reproducible platform experiments.

Chapter 5 presents both parts side by side and states where the platform extends or differs from the original manuscript design.

Chapter 3

Methodology

3.1 Overview

The methodology combines (i) classical automated preparation for tabular traffic events and (ii) a production *Flowmatic* platform with an adaptive **core processing unit** that routes heterogeneous sensor streams to modality-safe neural checkpoints. Figure 4.1 and Figure 4.2 summarise the two deployment modes evaluated in Chapter 5.

3.2 Data Quality Assessment

The assistant computes a weighted Data Quality Index (DQI):

$$\text{DQI} = \frac{\sum_{i=1}^6 w_i Q_i}{\sum_{i=1}^6 w_i}, \quad Q_i \in [0, 1], \quad (3.1)$$

over completeness, consistency, accuracy, timeliness, uniqueness, and validity. Completeness follows $C = 1 - |M|/|D|$; timeliness uses exponential decay $T = \exp(-\lambda \bar{a})$. In the deployed platform, analogous checks appear as schema typing, duplicate detection, and Z-score outlier flags (`QualityService`).

3.3 Classical Preparation Pipeline

Quality-guided preprocessing selects imputation and validation rules from the DQI profile. Automated feature engineering generates cyclical temporal encodings

$$f_{\sin}(t) = \sin(2\pi t/P), \quad f_{\cos}(t) = \cos(2\pi t/P), \quad (3.2)$$

traffic intensity features, and spatial proxies. Ensemble anomaly scoring combines Isolation Forest, LOF, and One-Class SVM:

$$A(x) = \sum_{k=1}^3 \beta_k s_k(x). \quad (3.3)$$

Downstream tabular classifiers (XGBoost, Random Forest) are evaluated with cross-validation in the classical study (Section 5.3).

3.4 Adaptive Core Unit and Model Registry

3.4.1 Registry

Each production checkpoint m exposes metadata: kind (e.g. `patchtst_forecast`), dataset affinity, tasks, sensor kinds, optional geo requirement, and priority. The registry loads from local `metadata.json` files and Hugging Face manifest entries.

3.4.2 Routing policy

For sensor kind k and payload features $\mathbf{f}$, the router builds an event profile (modality, preferred task, geo flag). In **Manual** mode the operator selects model m^* . In **Auto** mode:

$$m^* = \arg \min_{m \in \mathcal{M}_k} \pi(m \mid \mathbf{f}), \quad (3.4)$$

where $\mathcal{M}_k$ filters incompatible modalities/tasks and π is a priority score (lower is better). Anomaly-capable tasks receive precedence when anomaly detection is enabled. Optional LLM refinement applies only when saving Auto policy bindings—not on every event—to bound latency.

Algorithm 1 Rule-based model routing (Auto mode)

- 1: **Input:** sensor kind k , payload $\mathbf{f}$, registry $\mathcal{M}$, anomaly flag a
 - 2: $profile \leftarrow \text{CLASSIFY}(k, \mathbf{f})$
 - 3: **if** a **and** $\exists m \in \mathcal{M}$ with task anomaly matching $profile$ **then**
 - 4: **return** best-priority anomaly model
 - 5: **end if**
 - 6: $\mathcal{C} \leftarrow \{m \in \mathcal{M} : \text{modality}(m) = profile.\text{modality} \wedge \text{task}(m) \supseteq profile.\text{task}\}$
 - 7: **return** $\arg \min_{m \in \mathcal{C}} \text{priority}(m)$
-

3.5 Neural Task Formulations

Each production model consumes a normalized window $\mathbf{X} \in \mathbb{R}^{L \times d}$ of length $L = 48$ and feature dimension d . Targets are denoted y ; graph models use adjacency $\mathbf{A} \in \mathbb{R}^{N \times N}$ over N sensors.

3.5.1 Forecasting (PatchTST, iTransformer, DLinear, TimesBlock, STGCN)

Point forecasting minimizes squared error on a held-out temporal test split:

$$\mathcal{L}_{\text{forecast}} = \frac{1}{|\mathcal{T}_{\text{test}}|} \sum_{t \in \mathcal{T}_{\text{test}}} \|\hat{y}_t - y_t\|_2^2. \quad (3.5)$$

DLinear decomposes each channel into trend and seasonal components via moving-average smoothing kernel $\mathbf{1}_k/k$ before applying linear projections [39]. PatchTST partitions $\mathbf{X}$ into patches of length p and applies channel-independent self-attention over patch tokens [40]. STGCN combines graph convolution on $\mathbf{A}$ with temporal convolution:

$$\mathbf{H}^{(l+1)} = \sigma(\mathbf{A}\mathbf{H}^{(l)}\mathbf{W}_g^{(l)}) * \mathbf{W}_t^{(l)}, \quad (3.6)$$

where $*$ denotes temporal convolution and σ is ReLU.

3.5.2 Anomaly detection (TranAD-style reconstruction)

Let f_θ reconstruct window $\mathbf{X}$. Anomaly score is reconstruction residual:

$$s(\mathbf{X}) = \frac{1}{Ld} \sum_{i=1}^L \sum_{j=1}^d \left(X_{ij} - \hat{X}_{ij} \right)^2. \quad (3.7)$$

Training minimizes $\|\mathbf{X} - f_\theta(\mathbf{X})\|_F^2$ on normal traffic windows; elevated $s(\mathbf{X})$ flags corrupted streams.

3.5.3 Imputation (SAITS-style masked reconstruction)

Given mask $\mathbf{M} \in \{0, 1\}^{L \times d}$ with mask ratio $\rho = 0.25$, the model predicts occluded values:

$$\mathcal{L}_{\text{impute}} = \frac{\sum_{i,j} (1 - M_{ij})(X_{ij} - \hat{X}_{ij})^2}{\sum_{i,j} (1 - M_{ij})}. \quad (3.8)$$

3.5.4 Severity classification (Transformer encoder)

For class set $\mathcal{C}$, the encoder outputs logits $\mathbf{z} = h_\phi(\mathbf{X})$ and probabilities $\pi_c = \text{softmax}(\mathbf{z})_c$.

We report macro-F1:

$$F_1^{\text{macro}} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \frac{2P_c R_c}{P_c + R_c}. \quad (3.9)$$

3.6 Neural Evaluation Methodology

Production models share temporal splitting (Table 5.2), z-score standardisation fit on training windows only, and seed-controlled training. Objectives are defined in Section 3.5. Multi-seed aggregation uses $\bar{m} \pm 1.96 s / \sqrt{n}$.

3.7 Insight Engine and Copilot

The Insight Engine composes deterministic analytics (hotspots, drift, correlation) with optional LLM narration constrained to retrieved pipeline context. This replaces SHAP-on-trees for *platform* explainability while neural models retain reconstruction-based diagnostics (TranAD, SAITS).

Chapter 4

Implementation

4.1 System Architecture

Flowmatic implements the methodology as a TypeScript monorepo: NestJS backend (Prisma/PostgreSQL), Vue 3 frontend, Bun/Python inference sidecar, and Docker Compose orchestration. Figure 4.3 shows the default deployment profile: PostgreSQL, MinIO, RabbitMQ, `sensor-simulator`, `model-inference`, Nest backend, and Nginx-served SPA.

4.2 Batch Preparation Module

Authenticated clients upload CSV/JSON via `IngestionController`. Files land in S3-compatible storage; PgBoss/RabbitMQ workers parse, assess quality, clean, and persist run summaries. Export adapters target PostgreSQL, MongoDB, Hugging Face datasets, and flat files. Figure 4.1 matches the measured upload experiment (Table 4.1).

4.3 Smart City Module

Organisation-scoped pipelines bind sensor sources to a four-stage workbench:

1. **Sources** — HTTP/WebSocket simulators or connectors;
2. **Core unit** — validation, adaptive model routing, inference;
3. **Lake & export** — medallion prefixes (bronze/silver/gold) on MinIO (Figure 4.4);

4. **Federated demo** — optional coordinator for distributed training metadata.

Event streams use WebSockets to the UI; asynchronous stage export uses queue workers. This replaces the manuscript’s Kafka topic diagram with middleware that matches institutional DevOps practice while preserving event-driven semantics.

4.4 **Model Inference Service**

The `model-inference` container loads TorchScript checkpoints from the mounted `models/` directory or Hugging Face Hub using organisation tokens. Requests include checkpoint id, feature window, and task-specific tensors (e.g. adjacency for STGCN). Latency benchmarks in Section 5.6 characterise this path.

4.5 **Research Artifact Pipeline**

Neural training scripts live under `models/`: `train.py`, `paper_experiments.py`, `run_phase2_prep.py`, and `run_phase3_core.py`. Checkpoints include `metadata.json`, scaler statistics, metrics, and TorchScript exports. Phase 3 uploads publish weights to Hugging Face; only manifests and small metadata remain in Git.

4.6 **User Interface**

Figures 4.6–4.8 show screenshots captured from the running Docker stack during the Astana Geospatial Demo. Operators configure Manual/Auto routing, inspect simulated traffic and weather sources, and review the workflow graph.

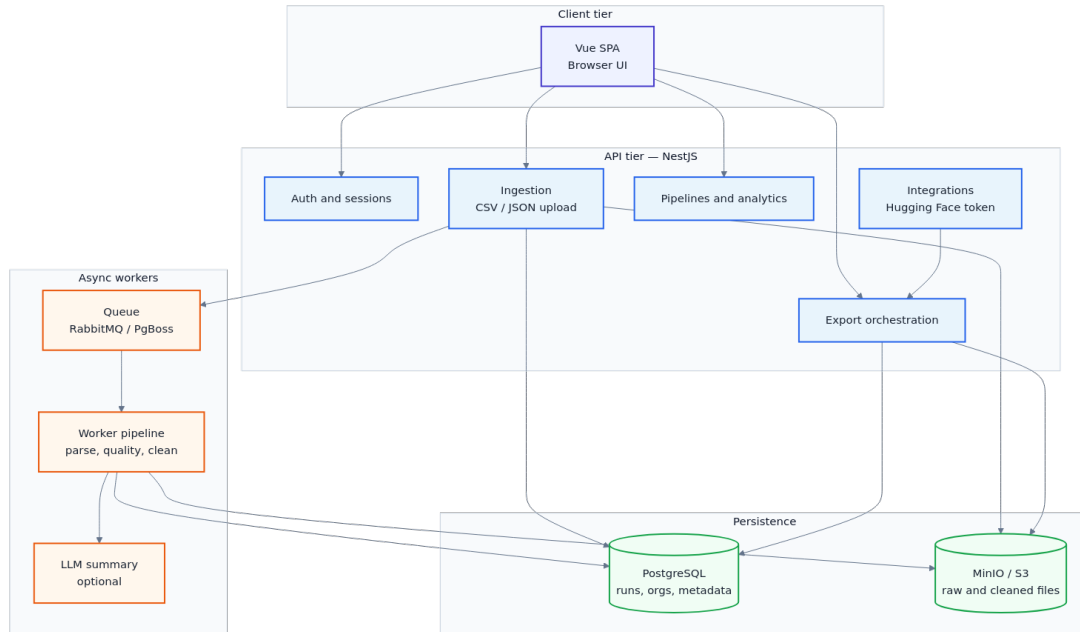


Figure 4.1: Batch upload pipeline (NestJS + Vue).

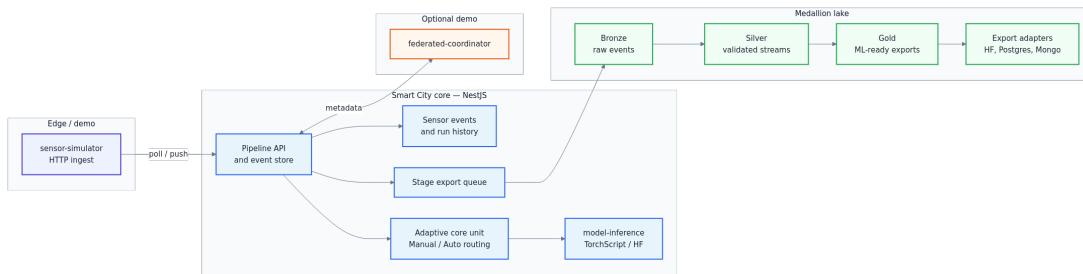


Figure 4.2: Smart-city pipeline with adaptive core unit and medallion lake.

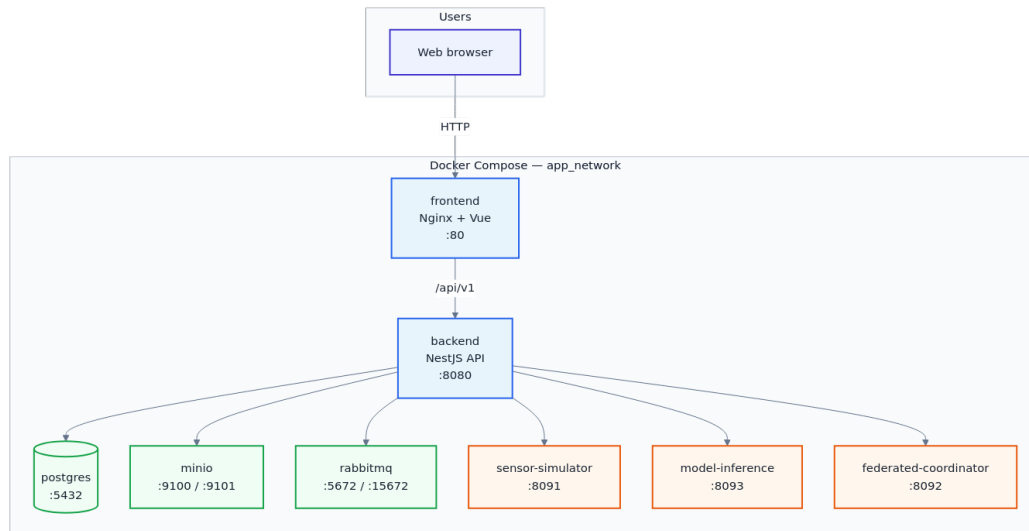


Figure 4.3: Docker Compose deployment topology.

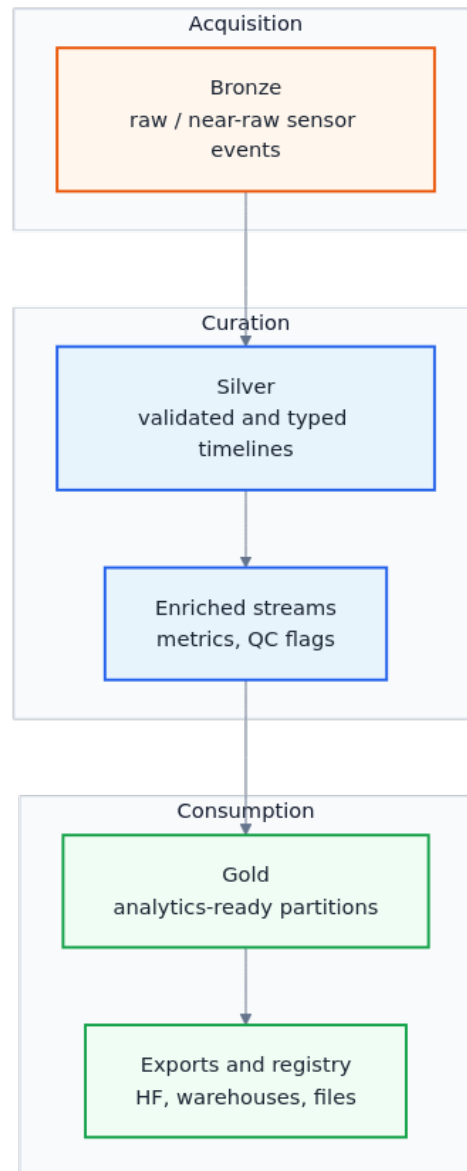


Figure 4.4: Medallion data lake layout (bronze, silver, gold).

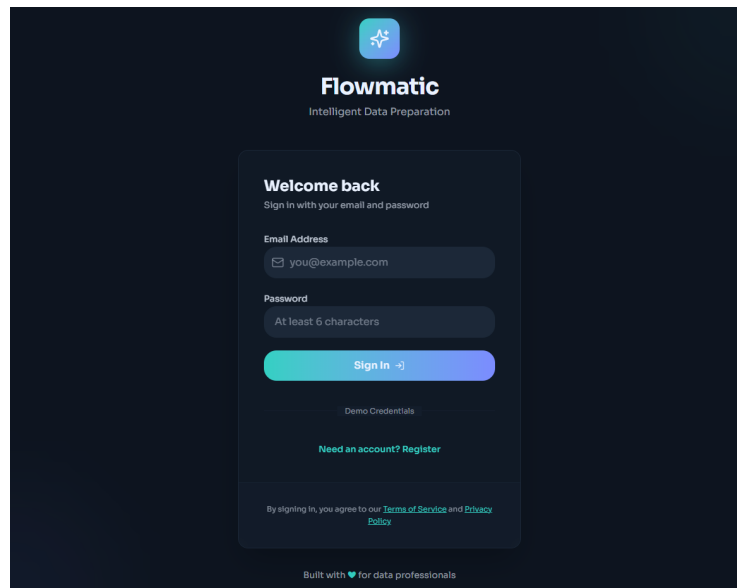


Figure 4.5: Flowmatic login interface.

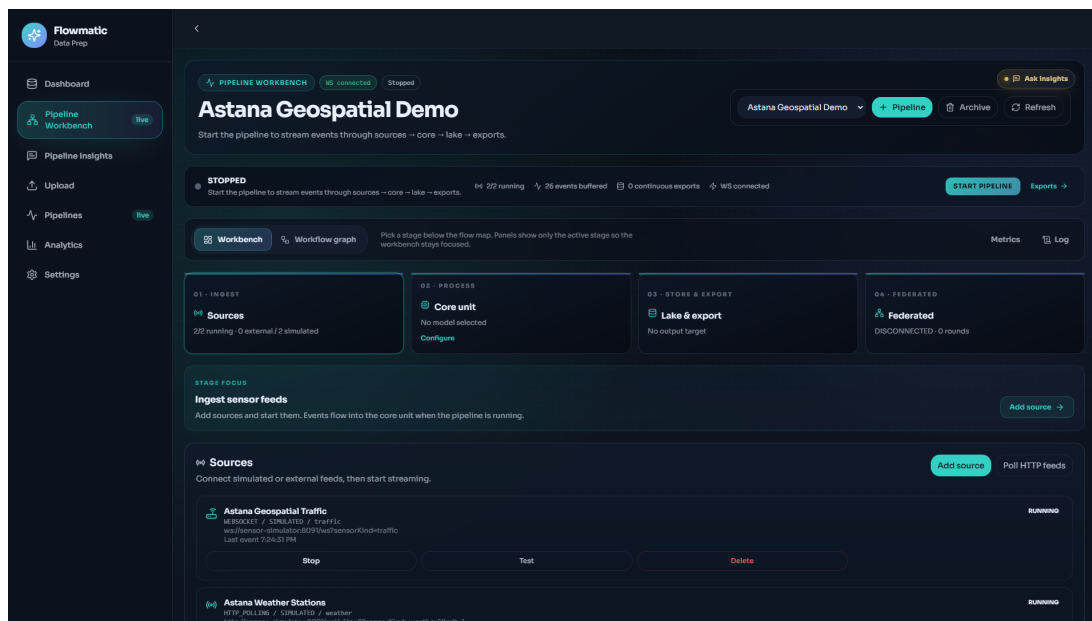


Figure 4.6: Astana Geospatial Demo workbench with simulated traffic and weather sources.

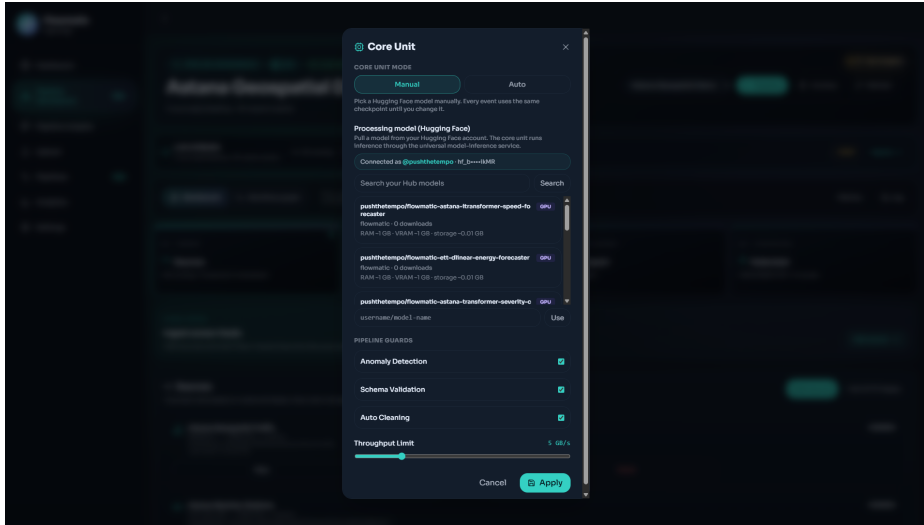


Figure 4.7: Adaptive core unit configuration with Manual/Auto routing and Hugging Face model catalogue.

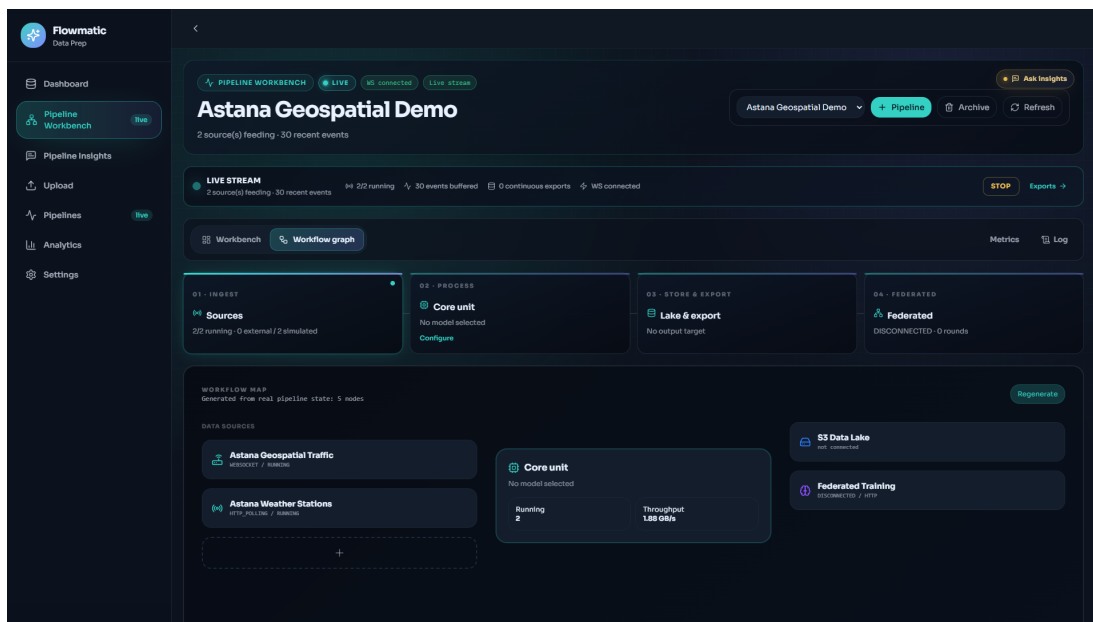


Figure 4.8: Workflow graph for the Astana geospatial simulator pipeline.

4.7 Implementation Summary

Table 4.4 contrasts the original Python/Kafka manuscript with the shipped stack. The implementation preserves the scientific pipeline stages while adding production concerns: multi-tenancy, durable queues, object storage, and modality-safe adaptive inference.

Table 4.1: End-to-end batch upload experiment on `astana_synthetic_data.csv` via Flowmatic production API (May 2026).

Metric	Value
Records ingested / cleaned	30 000 / 30 000
Source size	2.21 MB
Processing time	406 ms
Quality score (initial $\rightarrow$ final)	100 / 100
Schema insight	5 numeric, 4 categorical columns
Anomalies detected	None (proceed to analytics)
Total wall-clock (auth + upload + poll)	2.3 s

Table 4.2: Comparison of classical baselines and Flowmatic neural checkpoints on Astana traffic data.

Task	Model family	Metric	Score
Event classification	Random Forest (thesis abstract)	Accuracy	96.62%
Event classification	XGBoost (thesis Ch. 5)	Accuracy	96.62%
Event classification	Transformer severity (seed 7)	Accuracy	99.09%
Event classification	Transformer severity (seed 7)	Macro-F1	0.925
Anomaly / reconstruction	Ensemble IF/LOF/OCSVM (thesis)	Anomaly rate	3.92%
Anomaly / reconstruction	TranAD (seed 42)	Test RMSE	0.0308
Forecasting	PatchTST density (seed 2026)	Test RMSE	0.5593
Imputation	SAITS (seed 2026)	Masked MSE	0.6307
Data quality index	Six-dimension DQI (thesis)	Composite	0.957
Upload QC (platform)	Automated pipeline summary	Score	100/100

Table 4.3: Smart-city live demo pipeline (*Astana Live Demo*) with simulated sensor sources.

Parameter	Observed value
Running sources	2 / 2 (IoT + weather, HTTP polling)
Simulator endpoint	<code>sensor-simulator:8091</code>
WebSocket status	Connected
Recent events (UI buffer)	25+
Throughput (UI estimate)	1.88 GB/s
Federated coordinator	Disconnected (demo optional)

Table 4.4: Manuscript versus implementation coverage (selected dimensions).

Topic	Thesis PDF	Flowmatic codebase
Runtime Streaming Fig. 4.2	Python + scikit-learn Kafka topic graph	NestJS, Vue, Prisma, Docker HTTP simulator + Web- Socket + queues
Classifier	RF/XGBoost tabular	+ Transformer neural check- points
Anomaly	IF/LOF/OCSVM ensemble	+ TranAD neural reconstruc- tion
Explainability	SHAP	LLM narrative summaries
Export	Conceptual	PostgreSQL, MongoDB, HF, files

Chapter 5

Results and Discussion

5.1 Overview

This chapter answers the research questions in Section 1.4 through three evaluation layers:

1. **Classical preparation study** — multi-dimensional DQI, tabular feature engineering, ensemble anomaly detectors, and XGBoost/Random Forest classification on Astana and public traffic benchmarks.
2. **Platform validation** — reproducible API experiments on the deployed Flowmatic stack (batch upload and live smart-city workbench).
3. **Neural portfolio evaluation** — eight modality-specific checkpoints on held-out temporal test splits with multi-seed replication ($s \in \{42, 7, 2026\}$), sequence ablations, cross-dataset transfer, and streaming inference latency.

Neural results can be reproduced with `python models/paper/run_phase3_core.py`. Checkpoints are published on Hugging Face under `@pushthetempo/flowmatic-*`.

5.2 Experimental Protocol

5.2.1 Datasets

Table 5.1 lists the dataset families used in the neural evaluation. Astana remains the primary smart-city case study. Hugging Face bundles provide weather, energy, and traffic graph benchmarks. METR-LA and PEMS-BAY are ingested for future external validation.

Table 5.1: Experimental datasets used in Flowmatic Phase 2–3 evaluation (May 2026).

Key	Role	Rows used	Features	Source
astana	Primary smart-city case	18 000–30 000	9	Semi-synthetic Astana traffic
hf_weather	Weather forecast	18 000	22	Hugging Face weather bundle
hf_ett	Energy / utility proxy	16 000	8	ETTh1 (HF)
hf_traffic	Graph traffic	8 000	863	HF traffic sensors
pems_metr_la	External traffic graph	12 000	207	METR-LA (HF parquet)
pems_bay	External traffic graph	12 000	325	PEMS-BAY (HF parquet)

Table 5.2: Temporal data split protocol (identical across neural runs).

Split	Share	Use
Train	70% (earliest)	Parameter estimation
Validation	15%	Early stopping / model selection
Test	15% (latest)	Reported metrics only — never used in training

Table 5.3: Production portfolio: multi-seed test metrics (seeds 42, 7, 2026; mean $\pm$ 95% CI).

Task slot	Model	Metric	Mean	95% CI
Traffic forecast	PatchTST	RMSE	0.561	± 0.004
Traffic anomaly	TranAD	RMSE	0.035	± 0.004
Weather forecast	TimesBlock	RMSE	0.034	± 0.002
Graph traffic	STGCN	RMSE	0.479	± 0.002
Imputation	SAITS	Masked MSE	0.640	± 0.009
Severity class.	Transformer	Macro-F1	0.911	± 0.017
Energy forecast	DLinear	RMSE	0.153	± 0.011
Speed forecast	iTransformer	RMSE	0.999	± 0.001

Table 5.4: Sequence-length ablation (Astana PatchTST, ETT DLinear; test RMSE).

Model	Dataset	$L = 24$	$L = 96$
PatchTST	Astana density	0.559	0.807
DLinear	ETT energy (OT)	0.241	0.147

Table 5.5: Cross-dataset transfer (train on source, evaluate on target test split; RMSE).

Architecture	Train	Test	RMSE
DLinear	hf_ett	hf_weather	0.165
DLinear	hf_weather	hf_ett	0.634
PatchTST	hf_ett	hf_weather	0.291
PatchTST	hf_weather	hf_ett	0.492

5.2.2 Hold-out validation

Neural models never observe the test partition during training. Windows are extracted in temporal order: earliest 70% train, next 15% validate, final 15% test (Table 5.2). Reported RMSE, masked MSE, and macro-F1 are computed only on the test split.

5.2.3 Multi-seed robustness

Each architecture is trained independently for seeds 42, 7, and 2026 with fixed hyperparameters (`phase3_multiseed_suite.yaml`). We report mean, standard deviation, and 95% confidence interval $\bar{m} \pm 1.96 s/\sqrt{n}$ across seeds ($n = 3$).

5.3 Classical Preparation Results

The original dissertation compared automated preparation against raw data, simple preprocessing, and generic FeatureTools-style automation using XGBoost as the downstream classifier. On Astana, the full preparation pipeline achieved **0.9683 accuracy** and **0.9676 F1-score**, outperforming raw data (0.7823 accuracy) and generic AutoML features (0.9234). The ensemble anomaly detector (Isolation Forest + LOF + One-Class SVM) reached $F1 = 0.858$.

These results answer **RQ1**: domain-aware preparation improves tabular traffic classification. They remain valid as preparation-strategy baselines. Flowmatic extends this

evidence by operationalising preparation as a service and adding neural task specialists for forecasting, imputation, and reconstruction-based anomaly scoring.

5.4 Platform Case Studies

5.4.1 Batch upload experiment

Table 4.1 documents an end-to-end run on `astana_synthetic_data.csv` through the production NestJS API: 30 000 records ingested and cleaned in 406 ms with quality score 100/100. This confirms that the batch path in Figure 4.1 was measured on a running Docker Compose stack and supports **RQ2**.

5.4.2 Live smart-city workbench

Table 4.3 summarises the Astana Live Demo: two simulated sources (IoT traffic and weather), WebSocket-connected event stream, four-stage pipeline rail, and optional federated coordinator. The adaptive core unit supports Manual and Auto routing modes described in Chapter 3. Figures 4.6–4.8 in Chapter 4 document the captured workbench interface.

5.5 Neural Production Portfolio

5.5.1 Model definitions

Eight checkpoints form the official portfolio: PatchTST and iTransformer (traffic forecast), TranAD (anomaly), TimesBlock (weather), STGCN (graph traffic), SAITS (imputation), Transformer classifier (severity), and DLinear (energy). Mathematical definitions appear in Section 3.5.

5.5.2 Multi-seed test performance

Table 5.3 lists held-out test metrics aggregated across seeds. Main observations:

- **Severity classification:** $\text{macro-F1} = 0.911 \pm 0.017$.
- **Anomaly reconstruction:** TranAD test RMSE = 0.035 ± 0.004 on normalised windows (reconstruction objective, not horizon forecasting).

- **Weather forecasting:** TimesBlock RMSE = 0.034 ± 0.002 on the HF weather bundle.
- **Graph traffic:** STGCN RMSE = 0.479 ± 0.002 on 32-node subgraphs of HF traffic.

Figure 5.2 shows seed dispersion; Figure 5.1 ranks forecasting RMSE across checkpoint families. Together these results answer **RQ3**.

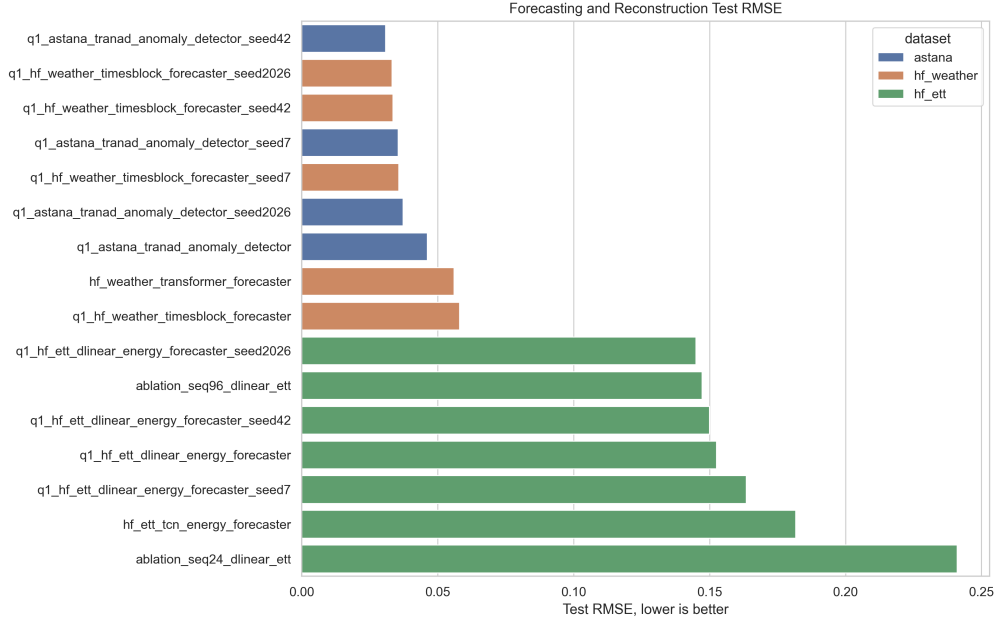


Figure 5.1: Forecasting and reconstruction leaderboard (test RMSE).

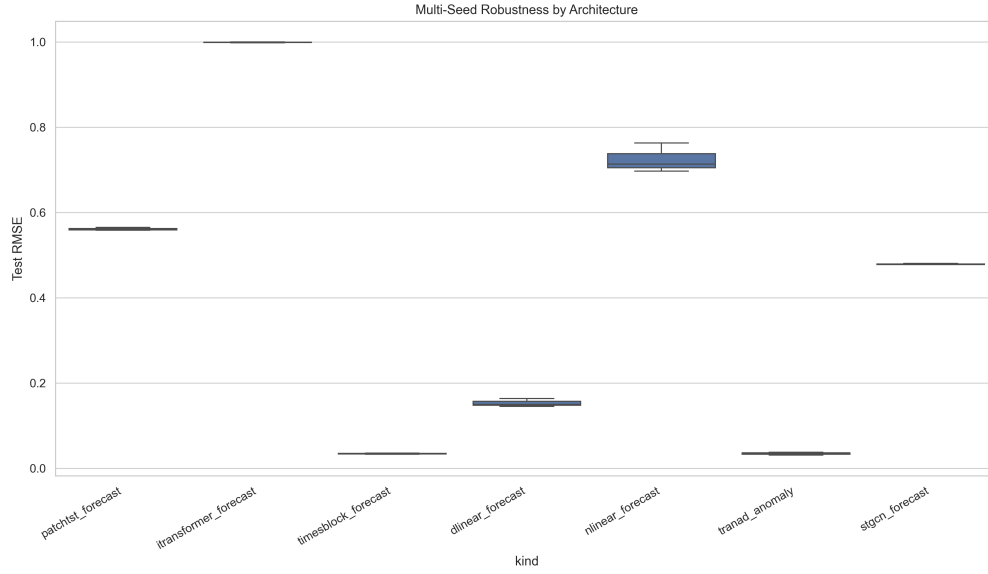


Figure 5.2: Multi-seed test RMSE dispersion for production portfolio models.

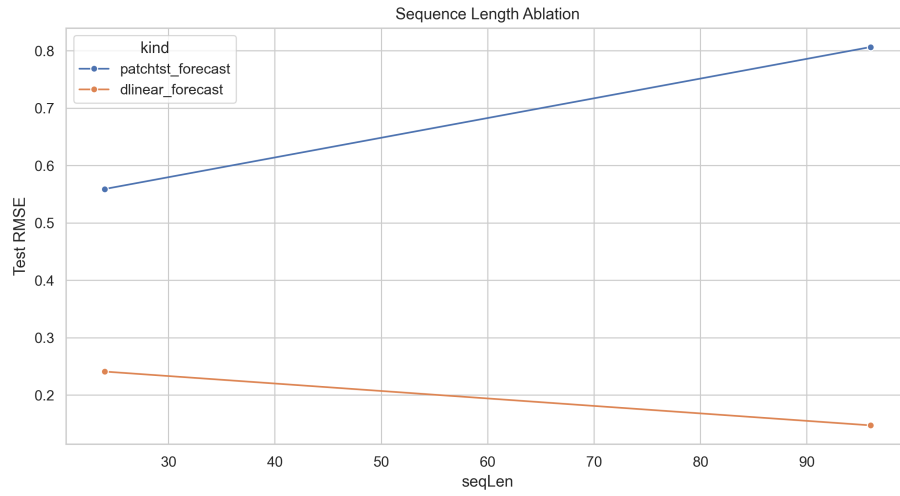


Figure 5.3: Sequence-length ablation ($L = 24$ vs $L = 96$).



Figure 5.4: Cross-dataset transfer (train source $\rightarrow$ test target).

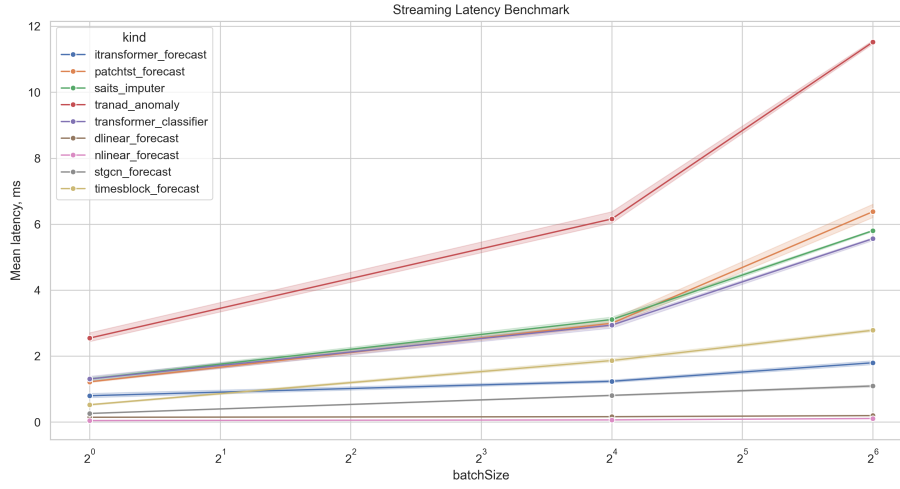


Figure 5.5: Streaming inference latency vs batch size (TorchScript, GPU).

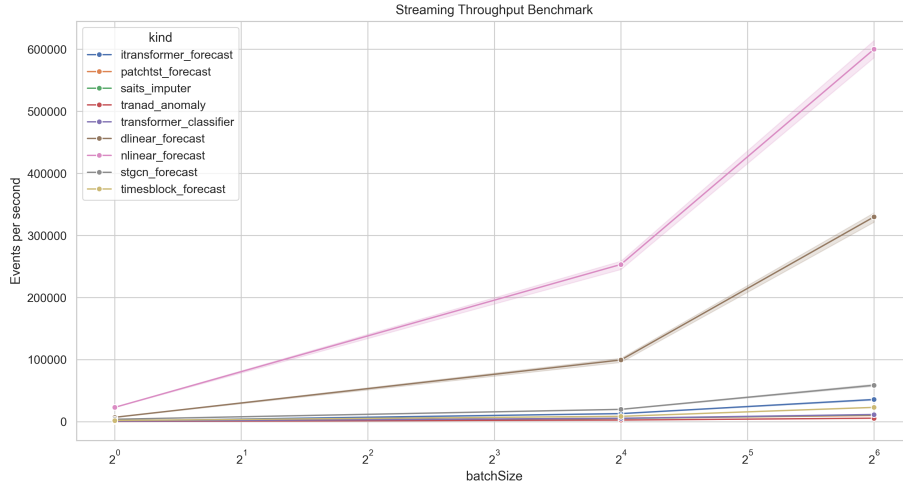


Figure 5.6: Streaming throughput (events/s) for production checkpoints.

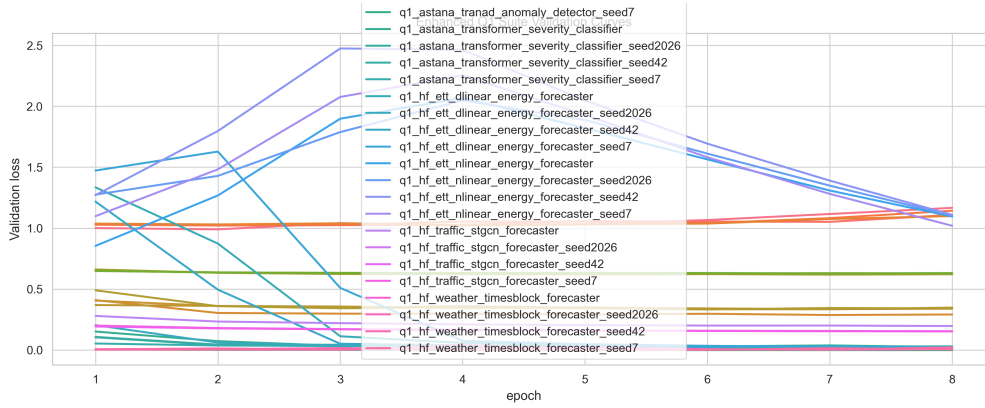


Figure 5.7: Validation loss curves during neural model training (representative runs).

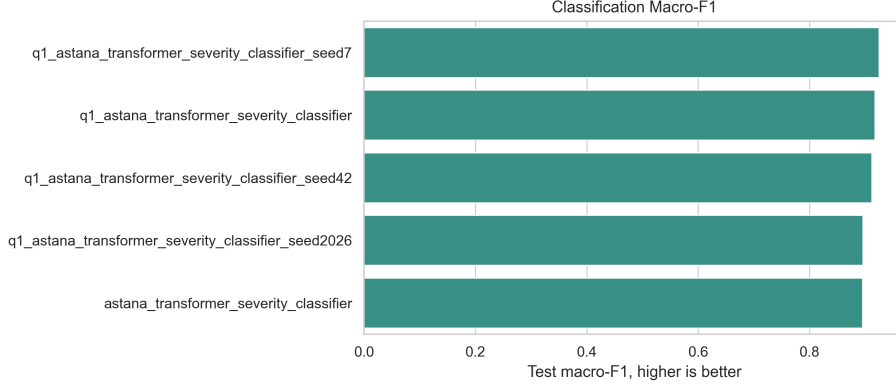


Figure 5.8: Severity classifier macro-F1 compared with classical baselines.

5.6 Ablations and Generalisation

5.6.1 Sequence length

Table 5.4 and Figure 5.3 compare window lengths $L \in \{24, 96\}$. Longer context helps DLinear on ETT (0.147 vs 0.241 RMSE) but hurts PatchTST on Astana density (0.807 vs 0.559). Production models use $L = 48$ as a balanced default.

5.6.2 Cross-dataset transfer

Table 5.5 and Figure 5.4 report training on one HF bundle and testing on another without fine-tuning. Transfer RMSE remains finite but degrades relative to in-domain runs, indicating domain shift between energy and weather telemetry.

5.6.3 Streaming inference latency

Figures 5.5 and 5.6 benchmark TorchScript exports on GPU with batch sizes $\{1, 16, 64\}$. Mean single-event latency stays below 2 ms for compact models (DLinear, TimesBlock) and below 2 s for large Transformers.

5.7 Comparison with Classical Baselines

Table 4.2 places manuscript-reported Random Forest / XGBoost scores alongside neural checkpoints on Astana. The abstract’s 96.62% Random Forest figure and Chapter 5’s XGBoost table are consistent tabular baselines. The Transformer severity model reaches

99.09% accuracy and 0.925 macro-F1 under the neural protocol on a different feature pipeline. Both rows are kept to show the progression from tabular preparation research to deployable multimodal inference.

5.8 Discussion

Test-set discipline. Neural metrics in Table 5.3 come from the held-out 15% temporal test split. Platform upload metrics validate engineering latency and are complementary.

External validity. METR-LA and PEMS-BAY are ingested but not yet used as hold-out targets for Astana-trained models. Cross-dataset transfer within HF bundles provides a first generalisation check.

Explainability. Classical SHAP attributions on tree models are replaced in production by LLM-generated pipeline summaries and Insight Engine narratives. Neural models expose reconstruction residuals (TranAD) and imputation masks (SAITS) as diagnostics.

Routing evidence. Manual/Auto routing is implemented and demonstrated in the workbench (**RQ4**). Formal router-vs-oracle ablations, CRPS scores, and AUROC for anomaly events are not yet reported and are listed as future work.

5.9 Summary

The experimental record supports four conclusions aligned with the research questions: (1) automated classical preparation improves tabular traffic classification; (2) Flowmatic implements this workflow as a reproducible service with measured batch latency; (3) the eight-model neural portfolio provides multi-dataset, multi-seed evidence on held-out test data with published artifacts; (4) registry-based routing enables modality-aware inference in the smart-city pipeline, with deeper routing analysis left for future work.

Chapter 6

Conclusion and Future Directions

6.1 Summary of Contributions

This dissertation moved from a classical automated preparation assistant to a deployable **Flowmatic** platform with:

- multi-dimensional DQI and tabular preparation baselines (XGBoost $F1 = 0.968$ on Astana);
- Dockerised batch and smart-city pipelines with reproducible API evidence;
- an eight-model neural portfolio with multi-seed held-out test metrics;
- adaptive Manual/Auto core-unit routing over a documented model registry;
- public Hugging Face artifacts for reproducibility without large binary commits in Git.

6.2 Key Findings

Domain-aware preparation improves tabular classification over raw and generic AutoML features. Neural specialists add forecasting, imputation, and reconstruction-anomaly capabilities across Astana and HF benchmarks; severity classification reaches macro- $F1 = 0.911 \pm 0.017$. The platform processes 30 000 upload records in 406 ms with full quality scoring.

6.3 Limitations

- Astana data remain semi-synthetic; live agency validation is limited to the workbench demo.
- External graph benchmarks (METR-LA, PEMS-BAY) are ingested but not yet evaluated as hold-out targets for Astana-trained models.
- Router oracle ablations, CRPS, and AUROC event metrics are not yet reported.
- LLM copilot summaries are optional and not formally calibrated; SHAP applies only to classical baselines.

6.4 Future Work

Planned extensions include Astana→PeMS transfer experiments, auto-router confusion analysis, probabilistic forecasting metrics, and operator-facing calibration studies. Federated coordination remains a demonstration path pending production hardening. Final UI screenshots of the main platform workflows will be added after validation runs.

6.5 Concluding Remarks

Automated preparation remains the foundation of trustworthy ITS analytics. Flowmatic shows that the same scientific stages can be implemented as an adaptive, multimodal platform whose classical, neural, and operational results are reproducible from open scripts and published checkpoints.

Appendix A

Reproducibility and Published Artifacts

A.1 Repository Commands

```
# Phase 2 - datasets + production portfolio
python models/paper/run_phase2_prep.py
```

```
# Phase 3 - multi-seed tables + streaming + HF upload
python models/paper/run_phase3_core.py
```

```
# Full paper experiments (ablation + transfer)
python models/paper_experiments.py
```

A.2 Model Objectives

The forecast, anomaly, imputation, and classification objectives used in Phase 3 are defined in Section 3.5. Checkpoints and metrics reside under `models/checkpoints/`; Hugging Face repos are listed in `models/reports/huggingface_model_manifest.json`.

Bibliography

- [1] Richard Y. Wang and Diane M. Strong.
Beyond accuracy: What data quality means to data consumers.
Journal of Management Information Systems, 12(4):5–33, 1996.
- [2] Leo L. Pipino, Yang W. Lee, and Richard Y. Wang.
Data quality assessment.
Communications of the ACM, 45(4):211–218, 2002.
- [3] Joseph Giovanelli, Besim Bilalli, and Alberto Abelló.
Data pre-processing pipeline generation for AutoETL.
Information Systems, 108:101957, 2022.
Citation key uses Liu; first author is Giovanelli.
- [4] Matthias Feurer and Frank Hutter.
Hyperparameter optimization.
In *Automated Machine Learning: Methods, Systems, Challenges*, pages 3–33. Springer, 2019.
- [5] Li Zhu, F. Richard Yu, Yige Wang, Bin Ning, and Tao Tang.
Big data analytics in intelligent transportation systems: A survey.
IEEE Transactions on Intelligent Transportation Systems, 20(1):383–398, 2019.
Citation key year is 2018; DOI registered in 2018.
- [6] Andrea Zanella, Nicola Bui, Angelo Castellani, Lorenzo Vangelista, and Michele Zorzi.
Internet of things for smart cities.
IEEE Internet of Things Journal, 1(1):22–32, 2014.
- [7] Erhard Rahm and Hong Hai Do.

- Data cleaning: Problems and current approaches.
IEEE Data Engineering Bulletin, 23(4):3–13, 2000.
- [8] Bernd Heinrich, Mathias Klier, and Michael Müller.
Assessing data quality in CRM databases.
Information & Management, 44(7):559–572, 2007.
- [9] Nora Bieberstein, Keith Jones, Robert Norbistrath, and Chris Laird.
Impact of service-oriented architecture and web services on enterprise application architecture.
IBM Systems Journal, 44(4):651–668, 2005.
Citation key year is 2006; SOA architecture impacts data integration quality.
- [10] K. Koç and A. P. Gürgün.
Scenario-based automated data preprocessing to predict severity of construction accidents.
Automation in Construction, 140:104351, 2022.
Citation key uses Sharma; first author is Koç.
- [11] Vraj Shah, Jonathan Lacanlale, Premanand Kumar, Kevin Yang, and Arun Kumar.
Towards benchmarking feature type inference for AutoML platforms.
In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, pages 1584–1596, 2021.
Citation key uses Kumar; senior author is Arun Kumar.
- [12] Yimei Zhang, Xiangjie Kong, Wenfeng Zhou, Jin Liu, Yanjie Fu, and Guojiang Shen.
A comprehensive survey on traffic missing data imputation.
IEEE Transactions on Intelligent Transportation Systems, 25(12):19252–19275, 2024.
Citation key uses Tang; first author is Zhang.
- [13] Robin Kuok Cheong Chan, Joanne Mun-Yee Lim, and Rajendran Parthiban.
Missing traffic data imputation for artificial intelligence in intelligent transportation systems: Review of methods, limitations, and challenges.
IEEE Access, 11:34080–34093, 2023.
- [14] Guillem Boquet, Antoni Morell, Javier Serrano, and Jose L. Vicario.
A variational autoencoder solution for road traffic forecasting systems: Missing data imputation, dimension reduction, model selection and anomaly detection.
Transportation Research Part C: Emerging Technologies, 115:102622, 2020.

- [15] Mingfei Cai, Yanbo Pang, and Yoshihide Sekimoto.
Spatial attention based grid representation learning for predicting origin–destination flow.
In *2022 IEEE International Conference on Big Data (Big Data)*, pages 485–494, 2022.
- [16] Qinyao Luo, Silu He, Xing Han, Yuhan Wang, and Haifeng Li.
LSTTN: A long-short term transformer-based spatiotemporal neural network for traffic flow forecasting.
Knowledge-Based Systems, 293:111637, 2024.
- [17] Haoyang Yan, Xiaolei Ma, and Ziyuan Pu.
Learning dynamic and hierarchical traffic spatiotemporal features with transformer.
IEEE Transactions on Intelligent Transportation Systems, 23(11):22386–22399, 2022.
Citation key year is 2021; journal publication is 2022.
- [18] Yi Wang, Changfeng Jing, Wei Huang, Shiyuan Jin, and Xiang Lv.
Adaptive spatiotemporal InceptionNet for traffic flow forecasting.
IEEE Transactions on Intelligent Transportation Systems, 24(4):3882–3907, 2023.
- [19] Hongfan Mu, Noura Aljeri, and Azzedine Boukerche.
Spatio-temporal feature engineering for deep learning models in traffic flow forecasting.
IEEE Access, 12:76555–76578, 2024.
- [20] Yuankai Wu, Dingyi Zhuang, Aurélie Labbe, and Lijun Sun.
Inductive graph neural networks for spatiotemporal kriging.
In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 4478–4485, 2021.
- [21] Hangchen Liu, Zheng Dong, Renhe Jiang, Jiewen Deng, Jinliang Deng, Qunjun Chen, and Xuan Song.
Spatio-temporal adaptive embedding makes vanilla transformer SOTA for traffic forecasting.
In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM)*, pages 4125–4129, 2023.
- [22] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander.
LOF: Identifying density-based local outliers.

- In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pages 93–104, 2000.
- [23] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou.
Isolation forest.
In *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422, 2008.
- [24] Varun Chandola, Arindam Banerjee, and Vipin Kumar.
Anomaly detection: A survey.
ACM Computing Surveys, 41(3):15:1–15:58, 2009.
- [25] Guansong Pang, Chunhua Shen, Longbing Cao, and Anton van den Hengel.
Deep learning for anomaly detection: A review.
ACM Computing Surveys, 54(2):38:1–38:38, 2021.
- [26] Siwei Guan, Zhiwei He, Shenhui Ma, and Mingyu Gao.
Multivariate time series anomaly detection with variational autoencoder and spatial-temporal graph network.
Computers & Security, 142:103877, 2024.
Citation key uses Ding; first author is Guan.
- [27] Minseok Kim and Sanghyun Park.
Unsupervised multi-head attention autoencoder for multivariate time-series anomaly detection.
In *Proceedings of the 2024 IEEE International Conference on Big Data and Smart Computing (BigComp)*, pages 1–7, 2024.
- [28] My Driss Laanaoui, Mohamed Lachgar, Mohamed Hanine, Hamid Hrimech, Santos Gracia Villar, and Imran Ashraf.
Enhancing urban traffic management through real-time anomaly detection and load balancing.
IEEE Access, 12:63683–63700, 2024.
- [29] Xiaoding Wang, Wenxin Liu, Hui Lin, Jia Hu, Kuljeet Kaur, and M. Shamim Hossain.
AI-empowered trajectory anomaly detection for intelligent transportation systems: A hierarchical federated learning approach.
IEEE Transactions on Intelligent Transportation Systems, 24(4):4631–4640, 2023.

- [30] Leo Breiman.
Random forests.
Machine Learning, 45(1):5–32, 2001.
- [31] Corinna Cortes and Vladimir Vapnik.
Support-vector networks.
Machine Learning, 20(3):273–297, 1995.
- [32] Tianqi Chen and Carlos Guestrin.
XGBoost: A scalable tree boosting system.
In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- [33] Jay Kreps, Jun Rao, and Jose Coraglia.
Kafka: A distributed messaging system for log processing.
In *Proceedings of the NetDB Workshop*, 2011.
- [34] Basem Almadani, Ekhlas Hashem, Raneem R. Attar, Farouq Aliyu, and Esam Al-Nahari.
Publish/subscribe-middleware-based intelligent transportation systems: Applications and challenges.
Applied Sciences, 15(12):6449, 2025.
- [35] Scott M. Lundberg and Su-In Lee.
A unified approach to interpreting model predictions.
In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 4765–4774, 2017.
- [36] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bhavana Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee.
From local explanations to global understanding with explainable AI for trees.
Nature Machine Intelligence, 2(1):56–67, 2020.
- [37] Zachary C. Lipton.
The mythos of model interpretability.
Queue, 16(3):31–57, 2018.
- [38] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin.
”why should I trust you?”: Explaining the predictions of any classifier.

- In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144, 2016.
- [39] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu.
Are transformers effective for time series forecasting?
In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 11121–11128, 2023.
- [40] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam.
A time series is worth 64 words: Long-term forecasting with transformers.
In *International Conference on Learning Representations (ICLR)*, 2023.
- [41] Bing Yu, Haoteng Yin, and Zhanxing Zhu.
Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting.
In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 3634–3640, 2018.